

Functions, Modules & Clean Code

The leap from a script that runs to a codebase that lasts. Functions with type hints, the difference between pure and impure code, modules and packages, and the refactor that turns 800 lines of mess into something a new engineer can read in ten minutes.

WHAT'S INSIDE

- 01** Anatomy of a clean function
- 02** Argument types in Python
- 03** Pure vs. impure functions
- 04** Modules, packages, main guard
- 05** Refactor: script → package

WHY CLEAN CODE

₹40 crore. The price tag of messy code.

BENGALURU FINTECH

₹40 Cr

Value created over 24 months by one rewrite.

DEFECT RATE

-63%

After one 800-line script became 14 small functions.

ONBOARDING

3w → 4d

From three weeks to four days for a new engineer.

OUTPUT

Identical

The product behaviour did not change. Only the code did.

LEARNING OUTCOMES

By the end of this lesson, you will be able to —

1

Write functions with type hints and one-line docstrings.

2

Distinguish positional, keyword and default arguments correctly.

3

Identify pure vs. impure functions on sight.

4

Structure a project as modules and packages.

5

Refactor a script into a testable, modular package.

6

Use the main guard so scripts are safe to import.

AGENDA

What we'll cover today

Short, sequenced parts. Each one builds on the last.

01

Function anatomy

The six signals that turn a function from 'works' into 'production-ready'.

02

Argument types

Positional, keyword, default, *args, **kwargs — when each one is the right call.

03

Pure vs. impure

Why purity is the cheat code for testable, parallel and predictable code.

04

Modules & refactor

Folders, packages, the main guard — and the actual mechanics of pulling a script apart.

01

PART 1 OF 4

Anatomy of a clean function

Six signals reviewers and interviewers look for — every one teachable in ten minutes.

SIX SIGNALS

What a senior engineer looks for, in five seconds

A clean function shows all six. A junior function usually misses three.



NAME

Verb-first

`calculate_score`, `send_email`, `validate_input`. Never `get_data` — what data?



TYPE HINTS

Every arg, every return

`def f(x: int, name: str) -> bool`. Reads like documentation, runs like code.



DOCSTRING

One line, plain English

"Return True if user is over 18 and verified." If you can't write it, the function is doing too much.



NO MUTABLE DEFAULTS

def f(items=None) — never []

Mutable defaults are evaluated once. They turn into the most surprising bug you'll ever debug.



SHORT

≤ 20 lines, one job

If you scroll inside one function, it is two functions wearing a trench-coat.



RETURN

Return, don't print

Functions that print can't be tested, can't be reused, can't be parallelised. Return values.

ARGUMENT TYPES

Five argument styles — and when to reach for each

Most functions need only the first two. The others are tools for specific jobs — use them sparingly, deliberately, and never to show off.

```
>>> PYTHON

# 1. positional      def add(a, b):          # required, in order
# 2. keyword         add(a=1, b=2)       # call-side clarity
# 3. default         def greet(name='friend'): ... # optional, with fallback
# 4. *args           def total(*nums):    # any number of positionals
# 5. **kwargs        def opts(**flags):   # any number of keyword args

# Best practice — combine, in this order
def search(query: str,
               *,
               limit: int = 20,
               verbose: bool = False) -> list:
    ...
```

→ Force keyword-only arguments with `\*` — it stops callers passing the wrong thing in the wrong slot.

02

PART 2 OF 4

Pure vs. impure — the secret to testable code

The single distinction that decides whether a function is easy or impossible to test.

THE DIVIDE

Pure functions are the gold standard — and you've already written some

A pure function: same input, same output, every time. No printing. No file reads. No global state. No surprises.

An impure function: anything that touches the outside world — logging, network, time, randomness, file IO. Useful, necessary, but expensive to test.

The senior move: keep computation pure, push impurity to the edges.

PURE — easy to test

```
def square(x): return x*x
```

Same input → same output. No side effects. Run a million times in parallel — works fine.

IMPURE — needs scaffolding

```
def log_score(x): print(x);  
save(x)
```

Touches the console, the filesystem and the clock. Tests need mocks, fixtures, cleanup.

HOW TO SPOT IT

Three quick checks to tell pure from impure

Apply these to any function in your code. If it fails one, that's where the test pain comes from.



CHECK 1

Same input → same output?

Call it twice with identical args. If results differ, something hidden is leaking in. Not pure.



CHECK 2

Any I/O at all?

`print()`, `open()`, `requests.get()`, `datetime.now()` — every one of these makes the function impure.



CHECK 3

Touches outside state?

Reads or writes a global, a class attribute, an env var — that's impurity by another name.

03

PART 3 OF 4

Modules, packages and the main guard

The folder structure every Python project deserves — once it grows past one file.

PROJECT LAYOUT

Module = file. Package = folder with an `__init__.py`.

The minimum viable structure for any project that's about to grow. Get this right on day one and you save weeks later.

```
>>> PYTHON
```

```
scoring/                                ← package (folder with __init__.py)
├── __init__.py                         ← marks this a package; can be empty
├── compute.py                         ← pure functions only
├── io.py                             ← reading & writing files
└── cli.py                             ← command-line entry point
```

```
tests/                                  ← always sibling to source
├── test_compute.py
└── test_io.py
```

```
# Inside cli.py – the main guard
if __name__ == '__main__':
```

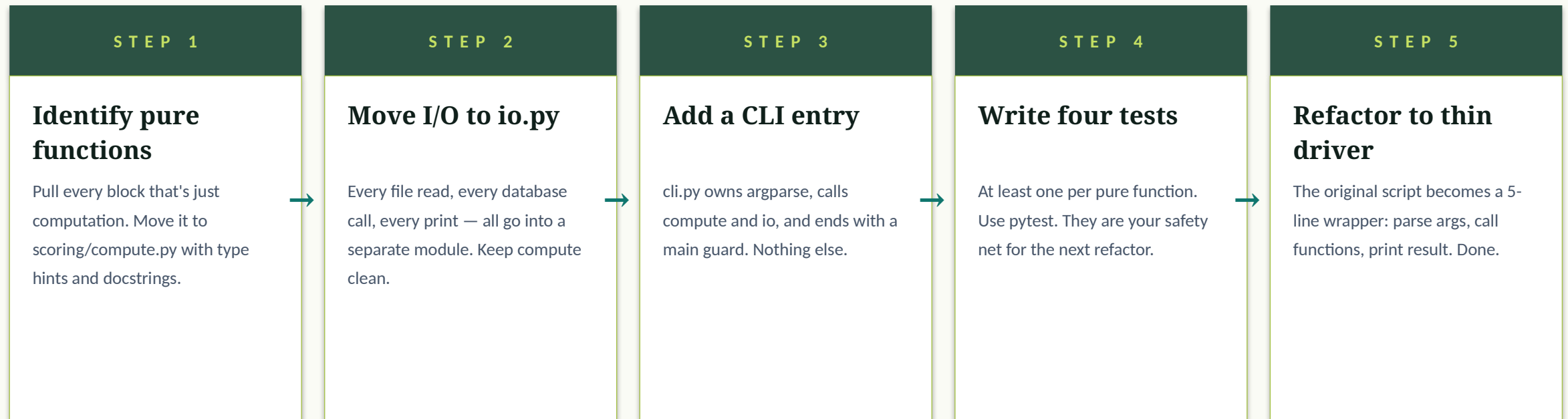
```
    main() # only runs when executed directly, not on import
```

→ The main guard is what makes a script safe to import. Without it, importing runs the whole file.

HOW TO DO IT

From 800-line script to modular package — five concrete moves

Done in this order, the refactor takes hours, not weeks. Skip a step and the codebase resists you.



04

PART 4 OF 4

The script-to-package refactor in practice

A worked example. Same behaviour, half the lines, one-tenth the cognitive load.

WORKED EXAMPLE

One 220-line student-grading script becomes a 5-file package

THE SCENARIO

A teaching assistant inherits a 220-line Python script that reads a CSV of student records, computes grades using six rules, writes a report and emails the principal. There are zero functions. Bug-fixes take half a day each. Three teachers refuse to touch it.

THE TAKEAWAY

Same behaviour, after the refactor: 5 files, ~120 lines of code, 8 pure functions, 12 tests. Fix-time drops from half a day to twenty minutes — and any of the three teachers can now do it.

WHAT TO NOTICE

- 1 Identify what is pure (the grading rules) and isolate it.
- 2 Move every IO call (CSV read, email send) to its own module.
- 3 Replace the 4 nested loops with two comprehensions and one map.
- 4 Add tests for every pure function — refactor with confidence.

✦ THINK • PAIR •
SHARE

ACTIVITY

Sort eight tiny functions into PURE / IMPURE

PROMPT

Decide whether each is pure or impure — and write the reason in one word.

1. `def square(x): return x*x`
2. `def now(): return datetime.now()`
3. `def add(a, b, log=True): if log: print(a,b); return a+b`
4. `def hash_id(x): return hashlib.md5(x).hexdigest()`
5. `def save_csv(data): df.to_csv('out.csv')`
6. `def parse_age(s): return int(s.split('=')[1])`
7. `def random_id(): return uuid.uuid4()`
8. `def total(nums): return sum(nums)`

HOW TO RUN IT

- 1 60s solo — write your answers in your notebook.
- 2 60s with a partner — argue the borderline cases.
- 3 60s share-out — instructor calls on one team.
- 4 30s reveal — instructor reveals the canonical sort.

Which function signature is the cleanest, most production-ready?

A

```
def calc(x, y, items=[])
```

B

```
def get_data(*args, **kwargs)
```

C

```
def calculate_score(records: list, *, weight: float = 1.0) -> float
```

D

```
def doStuff(records, weight)
```

Which function signature is the cleanest, most production-ready?



CORRECT ANSWER

calculate_score(records: list, *, weight: float = 1.0) -> float

WHY

Verb-first name, type hints on every argument and the return, a default that is immutable, and keyword-only protection on the optional argument. Reads as documentation. The other options break naming, hint, default-mutability or keyword-clarity rules.

Which of these is purest?

A

```
def read_users(): return open('users.csv').read()
```

B

```
def now_iso(): return datetime.now().isoformat()
```

C

```
def total(nums): return sum(nums)
```

D

```
def log_score(x): print(x); return x
```

Which of these is purest?



CORRECT ANSWER

```
def total(nums): return sum(nums)
```

WHY

Same inputs always produce the same output. No I/O, no globals, no clock. The other three each touch the outside world: filesystem, system clock, or stdout. Useful — but impure, and harder to test.

Why does every entry-point file end with `if __name__ == '__main__':`?`



CORRECT ANSWER

So the file's logic runs only when executed directly, not when imported

WHY

Without the guard, importing the module runs every top-level statement — which is a great way to email the principal twice by accident. The guard is the line that makes scripts importable, and importable scripts are testable scripts.

REFLECT & APPLY

Three questions before you close this lesson

Spend 60 seconds on each. Write the answers down — no scrolling, no shortcuts.

Q1

Pick one function in your last project. Could a new engineer name what it does in five seconds?

Q2

Where is the impurity in your code hiding? IO, time, randomness, globals — point to one and isolate it.

Q3

What is the smallest refactor you could ship this week — one impure function moved, one mutable default removed?

Refactor a real script into a modular package

Take a 100+ line script (yours, ours, or one from the internet) and refactor it into a package with at least four files: `__init__.py`, `compute.py`, `io.py`, `cli.py` — plus a `tests/` folder with four passing pytest cases.

DELIVERABLES

- Original script + final package side by side
- `compute.py` contains only pure functions, every one type-hinted
- `io.py` contains every file/network/print call
- `cli.py` uses `argparse` and ends with a main guard
- `tests/` with at least 4 pytest cases — all passing

WHAT 'GOOD' LOOKS LIKE

A great submission could be cloned from GitHub by a stranger tomorrow, and they could navigate the code in ten minutes. That is the bar.

SUMMARY

Five sentences to remember

If you remember nothing else from this lesson, remember these five lines.

- 1 Functions are sentences. Verb-first names, type hints, one job each.
- 2 Mutable defaults are evaluated once — they are the bug you'll only see in production.
- 3 Pure functions are the gold standard. Push impurity to the edges of the system.
- 4 A package is a folder with `__init__.py`. The main guard makes scripts importable.
- 5 The script-to-package refactor is five moves, not a rewrite. Done in order, it takes hours.

END OF LESSON 4

What you can now do

Each one of these is now part of your working vocabulary.

1

Spot a clean function vs. a beginner one in five seconds.

2

Pick the right argument style (positional, keyword, default) every time.

3

Tell pure from impure on sight — and isolate impurity.

4

Lay out a Python project in modules, packages and tests on day one.

5

Run the script-to-package refactor in five concrete steps.

NEXT → LESSON 5 — Setting Up Your Developer Environment